

Bài 7: CÂY NHỊ PHÂN TÌM KIẾM - NPTK

Mục tiêu:

Về kiến thức:

+ Nắm vững các khái niệm cơ bản về mô hình Cây nhị phân tìm kiếm, các phương pháp biểu diễn Cây nhị phân tìm kiếm.

Về kỹ năng:

+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên cây nhị phân tìm kiếm trong các bài toán cụ thể.

7.1 Định nghĩa cây nhị phân tìm kiếm

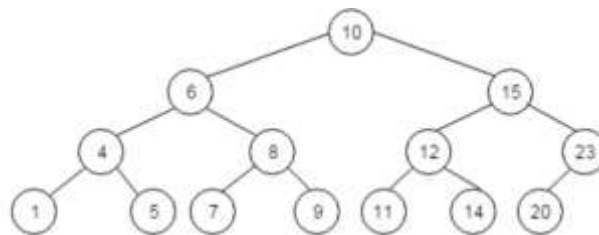
Cây nhị phân tìm kiếm là một cây nhị phân thỏa mãn các điều kiện sau:

Điều kiện 1: Tất cả các khóa tại các đỉnh của cây con bên trái đều có giá trị đi trước ($<$) các khóa tại đỉnh gốc

Điều kiện 2: Khóa tại gốc đi trước ($<$) tất cả các khóa ở các đỉnh của cây con bên phải

Điều kiện 3: Cây con bên trái và cây con bên phải cũng là cây tìm kiếm nhị phân

Ví dụ: về cây nhị phân tìm kiếm



Hình 7.1: Ví dụ về cây nhị phân tìm kiếm

Nhận xét:

- Dữ liệu trên mỗi nút có thể rất phức tạp. Trường khóa trên cây là kiểu có thứ tự có thể so sánh được.
- Trên cây NPTK không tồn tại hai nút có cùng khóa
- Cây con trái và cây con phải cũng là cây NPTK

- Khi duyệt cây NPTK theo thứ tự giữa sẽ thu được thứ tự các khóa theo thứ tự tăng dần.

7.2 Biểu diễn cây NPTK

Cây TKNP là cây nhị phân cho nên cây NPTK cũng có hai cách cài đặt bằng cấu trúc mảng và con trỏ như cây nhị phân. Tuy nhiên cần chú ý có thêm trường khóa tìm kiếm. Do trong cây nhị phân tìm kiếm còn có chứa hai thao tác là thêm và xóa nút trên cây nên chúng ta lựa chọn cài đặt cây nhị phân bằng con trỏ để thao tác thêm và xóa nút được linh hoạt hơn qua việc sửa lại các liên kết giữa các nút trên cây.

Mỗi nút trên cây sẽ cần quản lý thông tin về dữ liệu, khóa và hai con trỏ trỏ tới cây con trái và cây con phải. Như vậy mỗi nút trên cây là một bản ghi gồm có 4 trường: data, key, left và right được định nghĩa như sau:

```
#include<stdio.h>
#include<stdlib.h>
// định nghĩa cây nhị phân tìm kiếm bằng con trỏ
typedef struct Node
{
    int key;
    int data;
    Node *left, *right;
} Node;
typedef Node * SearchTree;
```

7.3 Triển khai các phép toán cơ bản trên cây NPTK

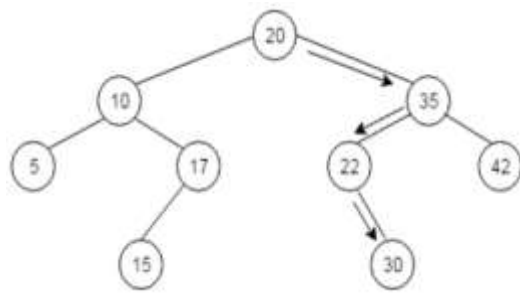
Vì cây NPTK là cây nhị phân nên nó cũng chứa các phép toán như trên cây nhị phân. Ngoài ra cây NPTK có thêm các phép toán sau:

7.3.1 Tìm kiếm một nút có khóa là x trên cây NPTK

- Thuật toán: Nếu cây khác rỗng ta tiến hành từ nút gốc bằng cách so sánh khóa của nút gốc với khóa x:

- + Nếu $x =$ khóa của nút gốc thì giải thuật dừng và hàm trả về vị trí của gốc.
- + Nếu $x >$ khóa của nút gốc thì ta tiến hành việc tìm khóa x trên cây con bên phải.
- + Nếu $x <$ khóa của nút gốc thì ta tiến hành việc tìm khóa x trên cây con bên trái.

- Ví dụ: Tìm khóa $x = 30$ trên cây NPTK sau:



- So sánh 30 với khóa nút gốc là 20, vì $30 > 20$ vậy ta tìm tiếp trên cây con bên phải, tức là cây có nút gốc có khóa là 35.
- So sánh 30 với khóa của nút gốc là 35, vì $30 < 35$ vậy ta tìm tiếp trên cây con bên trái, tức là cây có nút gốc có khóa là 22.
- So sánh 30 với khóa của nút gốc là 22, vì $30 > 22$ vậy ta tìm tiếp trên cây con bên phải, tức là cây có nút gốc có khóa là 30.
- So sánh 30 với khóa nút gốc là 30, $30 = 30$ vậy đến đây giải thuật dừng và ta tìm được nút chứa khóa cần tìm.

Hình 7.2: Ví dụ tìm khóa x trên cây NPTK

- Hàm tìm kiếm sau đây trả lại vị trí của nút có khóa là x , nếu không có hàm trả về NULL

```
// Viet phép toán tìm kiếm 1 khóa x trên cây tìm kiếm T
//input: T searchtree; x:int;
//output: Địa chỉ của nút có khóa là x Node;
Node * search(SearchTree T, int x)
{
    Node* V=T;
    int found=0;
    while ((V!=NULL)&& (found==0))
        if (x<V->key) V=V->left ;
        else if (x>V->key) V=V->right;
        else
            found=1;
    return V;
}
```

- Nhận xét: Thời gian thực hiện tìm kiếm $O(h)$, trong đó h là chiều cao của cây. Như vậy, để thuật toán thực hiện tìm kiếm nhanh thì cây nhị phân tìm kiếm cần phải được tổ chức tốt, cây cần phải tương đối “cân bằng”.

7.3.2 Thêm một nút có khóa x vào cây NPTK

- Bài toán đặt ra: Cho nút có khóa là x , và cây NPTK. Yêu cầu thêm khóa x vào cây NPTK sao cho sau khi thêm cây vẫn thỏa mãn điều kiện là cây NPTK

Theo định nghĩa cây tìm kiếm nhị phân ta thấy trên cây tìm kiếm nhị phân không có hai nút có cùng một khóa. Thuật toán như sau:

- Thuật toán: Trước hết ta phải tìm kiếm để xác định có nút nào chứa khóa x chưa?

+ Nếu có thì giải thuật kết thúc (không làm gì cả!).

+ Ngược lại, sẽ thêm một nút mới chứa khoá x này. Thêm khoá x vào cây TKNP đảm bảo cấu trúc cây TKNP không bị phá vỡ. Có nhiều cách để thêm, tuy nhiên để tránh phức tạp, người ta thường thực hiện thêm ở mức lá.

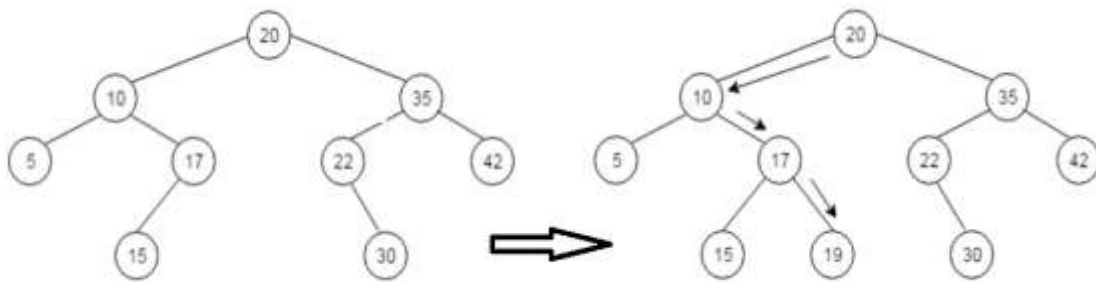
+ Nếu cây rỗng thì nút có khoá là x là gốc của cây.

+ Nếu cây $\neq$ rỗng, so sánh x với khoá ở gốc:

+ Nếu $x <$ khoá tại gốc, thêm nút có khoá là x vào cây con trái

+ Nếu $x >$ khoá tại gốc, thêm nút có khoá là x vào cây con phải

- Ví dụ: thêm khoá $x = 19$ vào cây NPTK



Hình 7.3: Ví dụ thêm khoá $x = 19$ vào cây NPTK

- Chương trình

```

void insertTree(SearchTree &T, int x)
{
    Node *V,*P,*N;
    int found=0;
    // cap phat 1 nut
    N=(Node*)calloc(1,sizeof(Node));
    if (N!=NULL)
    {
        N->left=NULL;
        N->right=NULL;
        N->key=x;
        if (T==NULL) T=N; // neu cay rong nut moi them chinh la goc cua cay
        else
        {
            V=T;
            while ((V!=NULL)&&(!found))
            {
                if (x<V->key )
                {
                    P=V;
                    V=V->left ;
                }
                else if (x>V->key )
                {
                    P=V;
                    V=V->right ;
                }
                else found=1;
            }
            if (found==0)
            {
                if (x<P->key) P->left =N; //them N vao ben trai cuar P
                if (x>P->key) P->right=N;
            }
        }
    }
    else printf("\n Ko cap phat dc bo nho");
}

```

7.3.3 Xóa một nút có khóa là x trong cây NPTK

Bài toán

Input: Cây nhị phân tìm kiếm gốc T, khoá x

Output: Nếu đỉnh có khoá x có trên T thì loại bỏ đỉnh này sao cho T sau khi loại bỏ x vẫn là cây TKNP

Các bước thực hiện:

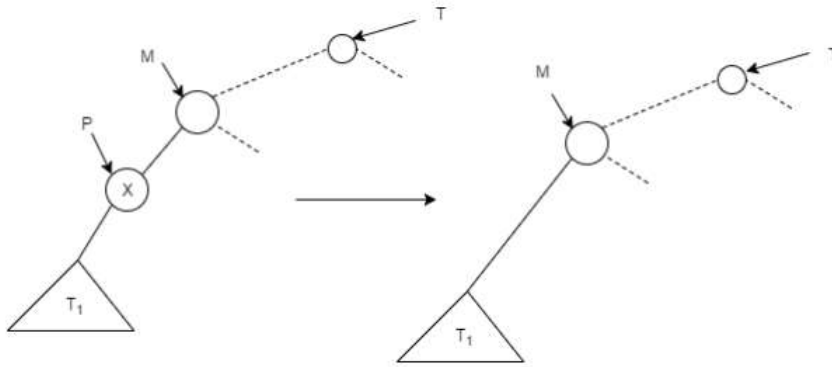
- Tìm đến vị trí nút có khóa là x. Nếu không tìm thấy nút chứa khoá x thì giải thuật kết thúc.

- Nếu khóa x có trên cây. Giả sử con trỏ P trở tới vị trí nút có chứa khoá x, M là

cha của P, ta có ba trường hợp sau:

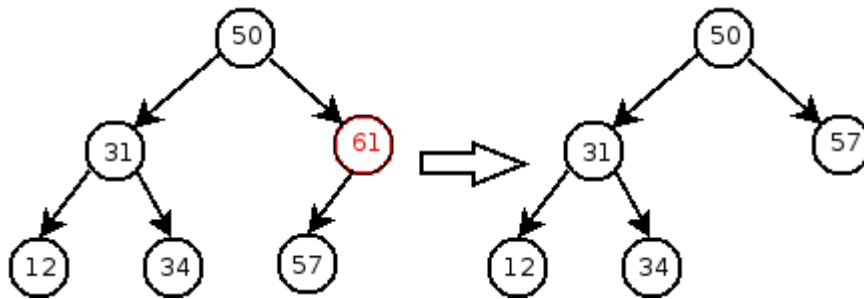
TH1: Nếu p là lá: ta xóa nút P và sửa lại liên kết $M \rightarrow \text{left} = \text{NULL}$ (nếu P là con trái của M) hoặc $M \rightarrow \text{right} = \text{NULL}$ (nếu P là con phải của M).

TH2: Nếu p có một con. Các thao tác như mô tả hình dưới đây:



Hình 7.4: Trường hợp P có 1 con trái (con phải của P rỗng) và P là con trái của M: $M \rightarrow \text{left} = P \rightarrow \text{left}$; $\text{free}(P)$; Các trường hợp khác tương tự. Như vậy trường hợp P có 1 con sẽ xảy ra 4 trường hợp. Ví dụ này là 1 trường hợp như mô tả

- Ví dụ:



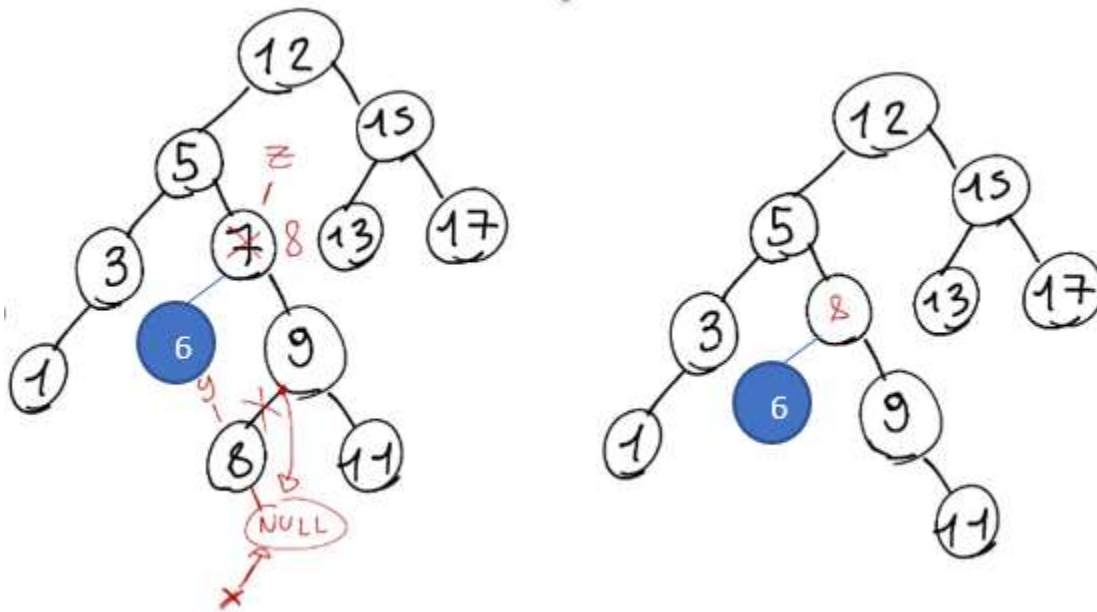
Hình 7.5: Ví dụ xóa khóa $x=6$ trên cây nhị phân tìm kiếm, trường hợp P chỉ có 1 con

TH3: P có 2 con đều khác rỗng:

Nguyên tắc: tìm khóa trên cây để thay thế P. Như vậy, khóa lớn nhất trên cây con trái của P (nút cực phải của cây con trái của P) hoặc khóa bé nhất trên cây con phải của P (nút cực trái của cây con phải của P) là hai khóa có thể thay thế vào P. Sau đó, xóa nút cực phải (hoặc nút cực trái).

Ví dụ: Xóa nút có khóa là 7 trên cây nhị phân tìm kiếm với trường hợp thay thế bởi

nút khóa nhỏ nhất trên cây con phải:



Hình 7.6: Ví dụ xóa $x = 7$ trong cây NPTK, trường hợp này 8 là nút nhỏ nhất trên cây con phải của P được thay thế vào P. Sau đó sửa lại các liên kết cho phù hợp.

- Trường hợp chọn nút lớn nhất trên cây con trái để thay thế làm tương tự.

7.4 Thực hành

7.4.1 Mục tiêu

Bài thực hành giúp sinh viên biết cách cài đặt cây nhị phân tìm kiếm theo con trỏ. Sau đó, sinh viên thực hành thành thạo trên các phép toán của cây nhị phân tìm kiếm. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng

7.4.2 Bài thực hành mẫu

Bài 1: Viết chương trình nhập vào cây nhị phân tìm kiếm có khóa là số nguyên. In cây vừa nhập

```
#include<stdio.h>
#include<stdlib.h>
typedef int item;
typedef struct Node
{
```

```

    int Key;
    item infor;
    Node *left;
    Node *right;
}Node;
//Dinh nghia cay
typedef Node *BinTree;
Node* Search(BinTree T, int x)
{
    Node *V=T;
    //int found=0;
    while (V!=NULL)
    {
        if (x==V->Key )
        {
            //found=1;
            return V;
        }
        else if(x<T->Key ) V=V->left ;
        else V = V->right ;
    }
    return V;
}
//Cap phat 1 Node N
//N->key =x
//N->left=NULL
//N->right=NULL
//Tim den vi tri them N sao cho thoa man cay NPTK
//Noi N vao vi tri thich hop
void Insert(int x, BinTree *T)
{
    Node *N, *V, *P; // V tim kiem; P luu vi tri cha cua V
    //V=*T;
    int found=0;
    // cap phat 1 o nho
    N=(Node *)calloc(1,sizeof(Node));
    if (N!=NULL)// neu cap phat thanh cong thi N!=NULL
    {
        printf("\n Cap phat thanh cong");
        N->Key =x;
        N->left =NULL;
        N->right =NULL;
    }
}

```

if (*T==NULL) *T=N;// kiểm tra xem cây rỗng không, nếu rỗng thì N chính gốc của cây

else// cây ban đầu khác rỗng

{

V=*T;

//tìm đến vị trí cần thêm

while ((V!=NULL)&&(!found))

{

if (x<V->Key)

{

P=V;// lưu địa chỉ của nút cha của V

V=V->left;

}

else if (x>V->Key)

{

P=V;

V=V->right ;

}

else found =1;

}

if (found ==0)

if (x<P->Key) P->left =N;//N nằm bên trái của P

else P->right =N;// N bên phải P

else printf("\n đã có x");

}

}

else printf ("\n Cap phát không thành công");

}

//duyet truoc: Cha-con trái - con phải

void Preorder(BinTree T)

{

if(T!=NULL)

{

printf("%5d",T->Key);// in nút cha

Preorder(T->left);

Preorder(T->right);

}

}

// duyet giua: con trái- cha - con phải

void ineorder(BinTree T)

{

if(T!=NULL)

{

```

        ineorder(T->left );
        printf("%5d",T->Key );// in du lieu cua nut cha ra man hinh
        //Preorder(T->left );
        ineorder(T->right );
    }
}
void postorder(BinTree T)
{
    if(T!=NULL)
    {
        postorder(T->left );
        //printf("%5d",T->Key );// in du lieu cua nut cha ra man hinh
        //Preorder(T->left );
        postorder(T->right );
        printf("%5d",T->Key );// in du lieu cua nut cha ra man hinh
    }
}
void NhapcayBinTree(BinTree *T)
{
    int x;
    *T=NULL;
    printf("\n Nhap x=");
    scanf("%d",&x);
    while (x!=-1)// for (int i=1; i<=n; i++)
    {
        Insert(x,T);
        printf("\n Nhap x=");
        scanf("%d",&x);
    }
}
int main()
{
    BinTree T;
    NhapcayBinTree(&T);
    Preorder(T);//Inorder(T);//Postorder(T);
}

```

Bài 2: Cài đặt cây nhị phân tìm kiếm có khóa là số nguyên sử dụng con trỏ. Thực hiện:

- Viết phép toán tìm kiếm một khóa trên cây NPTK
- Viết phép toán chèn một khóa vào cây NPTK
- Viết phép toán xóa một khóa trên cây NPTK

- Nhập vào cây NPTK cho đến khi nhập vào khóa là số nguyên âm
- In cây vừa tạo.
- Tìm phần tử lớn nhất trên cây
- Tìm phần tử nhỏ nhất trên cây
- Tìm số lẻ lớn nhất trên cây

Bài 3: An là người quan tâm đến thị trường chứng khoán. An theo dõi giá mở cửa và đóng cửa của N mã cổ phiếu yêu thích trong 10 ngày và ghi vào file data.txt có ví dụ như sau:

```
3
VIC 96.4 91.2
HSG 31.9 31.9
FLC 11.45 11.85
FLC 10.5 11.1
VIC 95.1 97.0
HSG 30.6 30.35
VIC 95.0 95.8
FLC 13.3 13.0
HSG 33.5 33.0
...
...
```

Các trường dữ liệu trong 1 dòng cách nhau bằng dấu \tab. Trong đó, dòng đầu tiên sẽ chứa thông tin về số mã cổ phiếu N (ví dụ N=3). N dòng tiếp theo chứa thông tin của N mã cổ phiếu trong ngày thứ 1 (ví dụ 3 dòng màu đỏ). Mỗi dòng trong N dòng này chứa thông tin mã cổ phiếu (mã cổ phiếu có 3 ký tự và chỉ gồm chữ cái và chữ số), giá mở cửa, giá đóng cửa ngày hôm đó. Tương tự như vậy với N dòng ứng với thông tin của các ngày thứ 2, thứ 3,..., thứ 10. Trong file data.txt, thứ tự các mã cổ phiếu không cần giống nhau giữa các ngày khác nhau.

Viết **menu** chương trình thực hiện các chức năng, chú ý khi thực hiện xong mỗi chức năng, chương trình quay lại menu chính:

Chức năng 1: Đọc file. Chương trình đọc file data.txt chứa giá cổ phiếu trong quá

khử và lưu vào RAM dùng cấu trúc dữ liệu phù hợp. Chương trình in thông tin ra màn hình tất cả các **mã chứng khoán theo thứ tự tăng dần** và **giá trị trung bình của hiệu (giá đóng cửa – giá mở cửa) trong 10 ngày** với mỗi mã đó. Yêu cầu: In ra giá trị trung bình với **3 chữ số sau dấu phẩy**.

Gợi ý: Sử dụng cây nhị phân tìm kiếm với khóa của nút là mã cổ phiếu, dữ liệu của nút là thông tin giá mở cửa và đóng cửa của 10 ngày.

Chức năng 2: Tìm kiếm theo mã chứng khoán. Chương trình cho phép người dùng nhập vào từ bàn phím 1 mã cổ phiếu. Nếu mã đó có trong cấu trúc dữ liệu hiện tại, chương trình in ra màn hình giá đóng cửa cao nhất và thấp nhất trong 10 ngày (**in với 3 chữ số sau dấu phẩy**). Nếu không có, chương trình sẽ thông báo ra màn hình là không tìm thấy.

Chức năng 3: Tìm kiếm những mã chứng khoán có xu hướng tăng. Trong 1 ngày, giá đóng cửa **lớn hơn** giá mở cửa là cổ phiếu tăng trong ngày đó. Chương trình in ra các mã tăng đồng thời trong cả ngày 1 và ngày 2.

Ví dụ: Trong file mẫu bên trên, mã FLC là kết quả cần tìm vì thỏa mãn tăng đồng thời cả ngày 1 (FLC 11.45 11.85) và ngày 2 (FLC 10.5 11.1). Trong khi đó mã VIC chỉ tăng trong ngày 2 mà không tăng trong ngày 1, nên không phải kết quả cần tìm.

Chức năng 4: Tìm mã có số ngày tăng lớn nhất. Chương trình in ra mã có số ngày tăng lớn nhất và số ngày lớn nhất đó. Nếu có nhiều hơn 1 mã có số ngày tăng lớn nhất giống nhau, hãy in ra tất cả các mã đó.

Chức năng 5: Thoát chương trình. Chương trình in ra thông tin tác giả (họ và tên sinh viên + MSSV) và sau đó thoát chương trình.